

Section 9: Other Optimizer Operators

1. Result Cache

- **Buffer Cache**와 혼동하면 안 됨
- **Result Cache**는 쿼리의 **최종 결과 자체**를 SGA에 캐싱하는 기능
- 기본적으로는 `RESULT_CACHE` 힌트나 테이블의 `MODE FORCE` 같은 설정을 통해 사용
- 같은 쿼리를 다시 실행할 때, 디스크/버퍼캐시에서 다시 읽고 집계하는 대신 **결과를 바로 재사용** 가능
- **Buffer Cache**는 디스크 대신 메모리에서 **data block** 을 읽게 해주는 캐시
- 즉 Buffer Cache는 블록 캐시이고, Result Cache는 결과 캐시

Buffer Cache = 원재료 캐시

Result Cache = 완성 결과 캐시

주의:

- Result Cache를 안 쓴다고 해서 캐시를 전혀 안 쓰는 건 아님
- 이 경우에도 Buffer Cache는 여전히 사용 가능

```
SELECT /*+ RESULT_CACHE */ cust_id, AVG(amount_sold)
FROM sales
GROUP BY cust_id
ORDER BY cust_id;

ALTER TABLE SALES RESULT_CACHE (MODE FORCE);
```

2. VIEW

- 실행계획의 `VIEW` 는 “저장된 **View 객체**를 사용했다” 는 뜻이 아니라, **별도의 query block(인라인뷰/서브쿼리)**이 따로 실행되었다 는 의미에 가까움
- 인라인 서브쿼리가 있다고 해서 무조건 `VIEW` 가 보이는 것은 아님
- 옵티마이저가 merge 가능한 경우, **join 형태로 풀어서 실행**할 수 있음

```
SELECT e.first_name, e.last_name, dept_locs_v.street_address
```

```

FROM employees e,
( SELECT d.department_id, l.street_address
FROM departments d, locations l
WHERE d.location_id = l.location_id ) dept_locs_v
WHERE dept_locs_v.department_id = e.department_id;

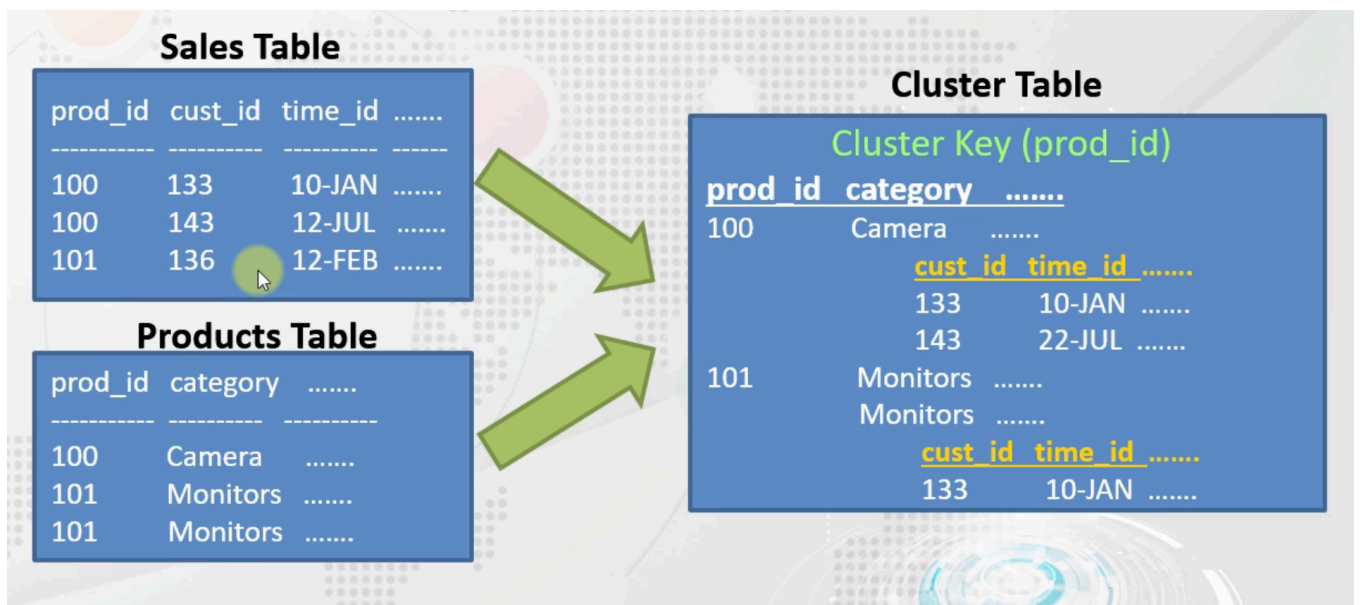
```

- 위 같은 인라인뷰도 merge 되면 실행계획에서 VIEW 가 안 보일 수 있음

추가:

- CREATE VIEW 로 만든 View도 단순하면 merge되어 기존 테이블 직접 접근처럼 보일 수 있음
- 즉 저장 View냐 인라인뷰냐보다, merge 되었느냐가 더 중요함

3. Cluster



- cluster는 같은 key 값을 가진 row들을 물리적으로 가깝게 저장해서 성능을 높이는 구조
- 특히 equi join (= 조건 조인)에 유리함

ex. department_id 기준으로 employees / departments 를 cluster에 저장하면,
같은 부서 데이터가 물리적으로 가까워져 조인 성능 향상

- 특히 hash cluster 는 equality search (=)에 강함
- 하지만 range scan (>, <, BETWEEN)이나 full scan 에는 불리할 수 있음

즉:

- = 조건 조인/조회에는 유리할 수 있음
- 전체 조회나 범위 조회가 많으면 오히려 손해 가능 (사용 주의)

```
select employee_id, first_name, department_name
from emp_clustered e, dept_clustered d
where e.department_id = d.department_id -- hash cluster 효과적
and e.department_id = 80;
```

```
select employee_id, first_name, department_name
from emp_clustered e, dept_clustered d
where e.department_id = d.department_id
and e.department_id > 80; -- hash cluster 비효율
```

변외) Partitioning

- 이건 cluster와 다르게 **대용량 테이블을 특정 기준(예: 날짜)**으로 나눠 저장하는 방식
- 날짜 조건 조회가 많을 때 **partition pruning**으로 필요한 파티션만 읽게 할 수 있음

정리:

- **Cluster** = 같이 자주 보는 row를 물리적으로 가깝게 저장
- **Partitioning** = 큰 테이블을 기준값별로 쪼개서 필요한 일부만 읽게 함

4. Sort / Hash 후처리

- DISTINCT, GROUP BY, ORDER BY 같은 작업은 후처리로 진행될 수 있음
- Oracle은 이를 **정렬 기반(SORT)** 또는 **해시 기반(HASH)**으로 처리할 수 있음

ex:

- DISTINCT → HASH UNIQUE 또는 SORT UNIQUE
- GROUP BY → HASH GROUP BY 또는 SORT GROUP BY

핵심:

- 정렬 결과가 굳이 필요 없으면 **hash** 방식으로 끝낼 수 있음
- **ORDER BY** 가 붙으면 결과를 정렬해야 하므로 **sort** 기반으로 기울기 쉬움
- 즉 불필요한 **ORDER BY**는 성능상 손해일 수 있음

5. INLIST

- **IN (...)** 조건이 있고, 해당 컬럼에 인덱스가 있으며 값 개수가 많지 않으면 **INLIST ITERATOR** 로 처리될 수 있음
- 느낌상 **for**문처럼 값을 하나씩 인덱스로 찾는 구조로 이해하면 됨

```
select * from employees where employee_id in (100, 110, 146);
```

결국 비용기반이므로 **IN** 값이 너무 많아지면 오히려 **Full Scan** 이 더 유리하다고 판단할 수 있음

6. UNION / MINUS / INTERSECT

집합 연산은 결과 집합끼리 비교해야 하므로 비용이 커질 수 있음

특히 **중복 제거 / 비교 작업** 때문에 **sort**가 발생할 수 있음

- **UNION** = 합치고 중복 제거
- **UNION ALL** = 그냥 합치기
- **INTERSECT** = 교집합
- **MINUS** = 차집합

핵심:

- **UNION** 보다 **UNION ALL** 이 보통 더 싸다
- **INTERSECT**, **MINUS** 는 경우에 따라 **EXISTS**, **NOT EXISTS**, **AND** 같은 형태로 바꾸면 더 효율적일 수 있음

